

Hardware/Software Co-Design for LLM Quantization

Comparing ASIC and FPGA Approaches for Efficient Inference

Gyanu Pandey

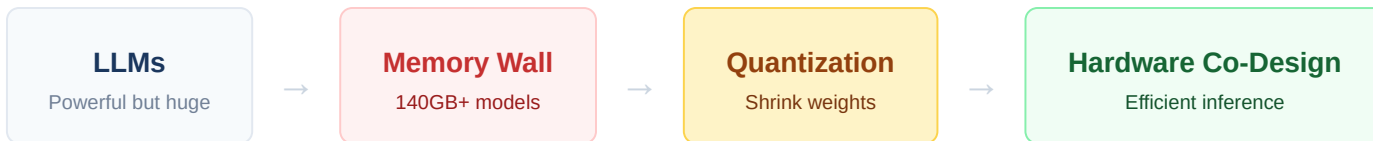
CECS 528 — Advanced Computer Architecture

December 2024

Introduction & Background

Large Language Models are Transforming AI

GPT-4, LLaMA, Claude — billions of parameters delivering unprecedented capabilities in text, code, and reasoning.



The Challenge

- Models too large for available memory
- High power consumption (300W/GPU)
- Inference cost at scale (\$700K/day)

The Opportunity

- Quantization reduces memory 2-4×
- Custom hardware eliminates overhead
- Co-design achieves both goals

This presentation: We analyze how hardware-software co-design approaches (Oltron, FlightLLM) outperform software-only quantization methods.

Motivation

Why Efficient LLM Deployment Matters

\$700K

per day

ChatGPT inference cost

300W

per GPU

A100 power consumption

140GB

model size

LLaMA-70B in FP16

Datacenter Impact

Thousands of GPUs running 24/7. Power costs dominate operating expenses. Every efficiency gain multiplies across the fleet.

Accessibility Gap

Consumer GPUs have 8-24GB. Most LLMs don't fit. Quantization + hardware co-design can democratize access.

Environmental

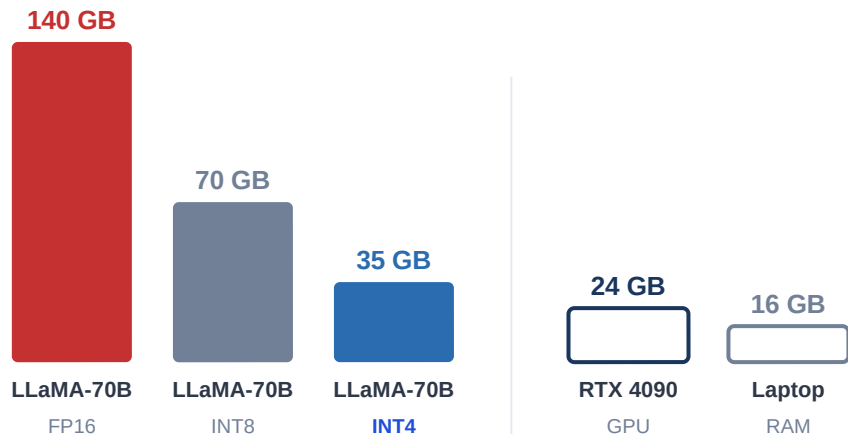
AI training and inference consume significant energy. Efficient deployment reduces carbon footprint at scale.

Bottom line: We need efficient deployment — not just accurate models. Hardware-software co-design addresses both.

The Problem: Memory Wall

LLaMA-70B requires **140 GB** of memory

Our laptop has 16 GB. Our GPU has 24 GB. The model doesn't fit.



How Quantization Helps

FP16 → **INT8**: 2× smaller

FP16 → **INT4**: 4× smaller

⚠ But: Naive quantization breaks model accuracy!

Key point: Quantization reduces precision to shrink model size — but we need smart approaches to maintain accuracy.

What is Quantization?

Reducing numerical precision of model weights and activations to decrease memory and computation.



Benefits

- Smaller model size
- Faster inference
- Lower power usage

Challenges

- Precision loss
- Outlier sensitivity
- Accuracy degradation

Key tradeoff: Lower precision means smaller size, but we risk losing accuracy if not done carefully.

Why Hardware Matters

Key insight: LLM inference is **memory-bound**, not compute-bound. The GPU sits idle waiting for data.

GPU Architecture Bottleneck



Thousands of GPU Cores (mostly idle)

BOTTLENECK

Memory Bandwidth

A100 GPU Specs:

Compute: 312 TFLOPS

Memory BW: 2 TB/s

LLaMA-70B: Read 140GB per token

Why Quantization Helps (But Not Enough)

Reduces memory traffic

Smaller weights = faster transfer

Software overhead

Dequantization at runtime costs cycles

Fixed GPU architecture

Cannot adapt to outlier patterns

Solution: Custom hardware for quantized inference

Software-Only Approach

Libraries like **bitsandbytes** let us run quantized models on standard GPUs

How Software Quantization Works



The **Dequantize** step is overhead — GPU converts before every operation

Popular Frameworks

bitsandbytes: Tim Dettmers, INT8/INT4

ONNX Runtime: Microsoft, optimized INT8

llama.cpp: Georgi Gerganov, CPU

GPTQ: Post-training quantization

Fundamental Limitation

GPU hardware is designed for **floating-point** operations. We store as INT8, but must convert back to FP16 for every computation. This overhead can exceed the memory savings!

Key point: Software approaches work, but cannot fully exploit quantization because the hardware is not designed for it.

Software-Only Limitations

Surprising finding: Quantization does not always speed things up — it depends on the implementation!

CipherCore Benchmark Results

Framework	Precision	Platform	Speed	Result
bitsandbytes	INT8	GPU	0.65x	35% SLOWER
ONNX Runtime	INT8	GPU	1.7x	Good speedup
llama.cpp	INT4	CPU	0.3x	CPU limited

Why the Slowdown?

Dequantization overhead on every operation

Memory access patterns not optimized

GPU cores designed for FP, not INT

Overhead can exceed memory savings

The Core Problem

Software runs on **fixed hardware**. No matter how clever our code, we are limited by what the GPU can physically do. We need hardware designed for quantized operations.

Bottom line: Software-only approaches cannot fully exploit quantization — the hardware is not designed for it.

Research Focus

How do hardware-assisted quantization approaches compare to software-only methods in terms of performance and energy efficiency?

Approaches Compared

Software-Only

- bitsandbytes, ONNX, llama.cpp

Oltron (ASIC)

- DAC 2024, Peking University

FlightLLM (FPGA)

- FPGA 2024, Tsinghua

Evaluation Metrics

- Throughput, Energy efficiency, Accuracy

Hypothesis

Hardware-assisted approaches will outperform software by eliminating dequantization overhead.

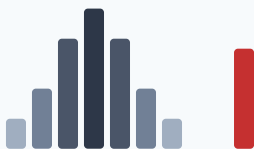
Oltron: ASIC Approach

Algorithm-Hardware Co-design for Outlier-Aware Quantization

DAC 2024 · Peking University · Custom ASIC Accelerator

The Outlier Problem

Activation Value Distribution



Scale for normal values?

Normal 0.5 $\rightarrow$ 64 ✓ | Outlier 100 $\rightarrow$ OVERFLOW! ✗

Scale for outliers?

Outlier 100 $\rightarrow$ 127 ✓ | Normal 0.5 $\rightarrow$ rounds to 1 ✗

Oltron's insight: Outliers aren't random — they cluster in specific channels, and patterns vary by layer. Design hardware that **adapts** to each layer's outlier distribution.

Approach: Algorithm finds optimal outlier ratio per layer $\rightarrow$ Reconfigurable hardware adapts accordingly

Oltron: Key Innovations

Three Core Innovations

INNOVATION 1

Adaptive Quantization

Analyze each layer's outlier distribution. Allocate INT4 vs FP16 precision per-channel based on sensitivity.

Layer 1: 2% FP16 | Layer 5: 15% FP16

INNOVATION 2

Reconfigurable PEs

Processing elements dynamically switch between INT4 and FP16 operations at runtime.

Hardware adapts to each layer's needs

INNOVATION 3

Tile-Based Dataflow

Reorganize memory access patterns to maximize data reuse and minimize bandwidth requirements.

Reduces memory bottleneck

Co-Design Principle: Algorithm determines optimal outlier ratio → Hardware reconfigures to match → Both work together for maximum efficiency

1.9×

Speedup

1.6×

Energy Efficiency

<1%

Accuracy Loss

12nm

Process Node

Oltron: Results

Performance vs OliVe (Previous Best)

Metric	OliVe	Oltron	Improvement
Performance	1.0x	1.9x	1.9x faster
Energy Efficiency	1.0x	1.6x	1.6x better
Accuracy Loss	Baseline	Less than 1%	Maintained

Implementation

Process 12nm	Clock 500 MHz	Models LLaMA, OPT
------------------------	-------------------------	-----------------------------

Why These Results?

Adaptive Quantization

Each layer optimized individually

Hardware Flexibility

No wasted compute cycles

Memory Efficiency

Tile-based layout reduces overhead

Co-Design Synergy

Algorithm + hardware working together

Takeaway: 1.9x speedup and 1.6x energy efficiency with less than 1% accuracy loss — the power of co-design

FlightLLM: FPGA Approach

Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs

FPGA 2024 · Tsinghua and Shanghai Jiao Tong University · Xilinx U280

Why FPGA?

GPU Limitations

Fixed architecture: Cannot skip zero computations

Memory bottleneck: Must fetch from DRAM every token

High power: A100 draws 300-400W

FPGA Advantages

Reconfigurable fabric: Design custom compute paths

On-chip SRAM: 100x faster than DRAM

Lower power: Around 75W for similar work

FlightLLM insight: Design FPGA architecture specifically for sparse, quantized LLM inference — exploit sparsity, keep data on-chip, compile once and run any sequence length.

Target: Efficient decode phase — the bottleneck in real LLM serving (one token at a time)

FlightLLM: Key Innovations

Three Core Innovations

INNOVATION 1

Configurable Sparse DSP

Custom DSP units that detect and skip zero-value computations entirely. No wasted cycles.

1.6× compute efficiency gain

INNOVATION 2

Always-On-Chip Decode

KV cache stored entirely in fast SRAM instead of slow DRAM. Eliminates memory bottleneck.

100× faster memory access

INNOVATION 3

Length-Adaptive Compile

Single compilation works for any sequence length. No recompilation needed for different inputs.

Handles 128 to 2048+ tokens

FPGA Advantage: Reconfigurable fabric enables sparsity exploitation + on-chip memory optimization — impossible on fixed GPU architecture

6×

Energy Efficiency

1.2×

vs A100 Throughput

75W

Power Draw

\$5K

Xilinx U280

FlightLLM: Results

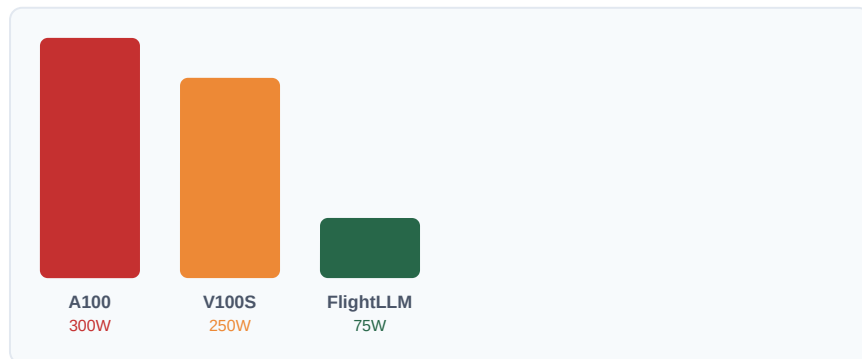
Performance Comparison

Platform	Power	Throughput	Energy Eff.
V100S GPU	250W	1.0x (baseline)	1.0x
A100 GPU	300W	1.1x	0.9x
FlightLLM	75W	1.2x	6x better

Implementation

Hardware Xilinx U280	Memory 8GB HBM2	Model LLaMA-7B INT4
--------------------------------	---------------------------	-------------------------------

Power Comparison



Bottom line: Same throughput as \$15,000 A100, at **1/4 the power**, on a \$5,000 FPGA.

Takeaway: FPGA delivers GPU-level throughput at 1/4 the power — critical for sustainable deployment at scale

Comparison: Software vs Hardware

Approach	Platform	Key Technique	Speedup	Energy Eff.
Software-Only	Standard GPU	Dequantize at runtime	0.65-1.7x	1x baseline
Oltron (ASIC)	Custom accelerator	Adaptive quantization	1.9x	1.6x
FlightLLM (FPGA)	Xilinx U280	Sparsity + on-chip mem	1.2x vs A100	6x

Software-Only Limitations

Fixed hardware cannot adapt to outliers

Dequantization overhead on every op

Cannot exploit sparsity efficiently

Hardware-Assisted Advantages

Native INT4/INT8 compute units

Reconfigurable for different layers

Memory architecture co-optimized

Trade-off: Software approaches are more accessible (standard GPUs), but hardware-assisted solutions deliver significantly better efficiency for production deployment

Conclusion: Co-design approach (algorithm + hardware together) consistently outperforms software-only solutions

Future Work

1 Hands-On Implementation

Benchmark software quantization tools (bitsandbytes, llama.cpp) on local hardware to validate CipherCore findings

2 Emerging Hardware

Evaluate newer accelerators: Groq LPU, AWS Inferentia, Google TPU — all natively support quantized inference

3 Cost-Benefit Analysis

Total cost of ownership: GPU rental vs FPGA deployment vs custom ASICs at datacenter scale

Additional Research Direction

Combine Oltron's adaptive quantization with FlightLLM's sparsity exploitation — could yield even greater efficiency gains by addressing both outliers and zeros simultaneously

Course Connection

These hardware approaches exemplify the memory hierarchy optimization, dataflow architecture, and hardware-software co-design concepts from our Advanced Computer Architecture curriculum

Conclusion

Three Key Takeaways

1 Hardware Matters

LLM inference is memory-bound. Custom hardware eliminates dequantization overhead that limits software approaches.

2 Co-Design Wins

Algorithm + hardware designed together — Oltron's adaptive quant + reconfigurable PEs, FlightLLM's sparse DSPs.

3 Energy is Key

FlightLLM's 6x efficiency shows why specialized accelerators matter for sustainable AI deployment at scale.

Key Numbers to Remember

140 GB

LLaMA-70B memory

1.9x

Oltron speedup

6x

FlightLLM efficiency

75W

vs 300W GPU

References: Oltron (DAC 2024, Peking University) · FlightLLM (FPGA 2024, Tsinghua and SJTU) · CipherCore Benchmark Study

Any Questions?

Hardware/Software Co-Design for LLM Quantization

1.9x

Oltron Speedup

6x

FlightLLM Efficiency

75W

vs 300W GPU

Gyanu Pandey · CECS 528 · December 2024